

CS 668 LLM Final Project Report

I.) Goal and Motivation

The goal of this project is to build an MLB analytics assistant that can answer baseball questions in natural language. Instead of needing to manually search through large baseball datasets, a user can ask questions like, “Who had the most home runs in May?” or “How did Judge do against sliders?” and quickly get an answer based on real MLB data.

My main motivation for this project is the idea of helping MLB announcers, analysts, and broadcasters find useful information faster. Before games, announcers usually prepare many notes and statistics, but baseball is unpredictable. During a game, certain situations can happen where a very specific stat becomes useful. For example, if a hitter is facing a certain pitcher, or if a player is struggling against a certain pitch type, the announcer could quickly ask the system for that information instead of searching manually. This kind of tool could help announcers add more precise context to what is happening on the field.

II.) Data Sources

1. Statcast 2025 season data
 - pitch-level and play-level data
 - used for time filtering, pitch type analysis, matchups, and custom stats
2. batting_stats_bref(2025)
 - official full-season hitter stats
 - used for hitter names, IDs, and official season totals
3. pitching_stats_bref(2025)
 - official full-season pitcher stats
 - used for pitcher names, IDs, ERA, WHIP, IP, SO/9, wins, and losses

III.) Data Cleaning and Preprocessing

The cleanData function prepares the raw Statcast data for analysis. First, it creates a copy of the original dataframe and checks that the required columns are present. It then filters the data to regular season games only, since the project focuses on the 2025 MLB regular season. The function also creates a cleaned event column called events_clean, which standardizes event names by filling missing values, converting text to lowercase, and removing extra spaces. This makes it easier to classify baseball outcomes consistently.

After that, the function defines which events count as plate appearances, official at-bats, hits, walks, strikeouts, and total bases. For example, singles, doubles, triples, and home runs count as hits, while walks and hit-by-pitches count as plate appearances but not official at-bats. The function also handles special cases such as `strikeout_double_play` and excludes `truncated_pa` because it does not represent a completed plate appearance. Also, the function adds new numeric columns such as `is_plate_appearance`, `is_at_bat`, `is_hit`, `is_home_run`, `total_bases`, `is_walk`, and `is_strikeout`. These columns allow the later tools to calculate player statistics by grouping and summing the data.

IV.) Tools Built

Tool	Purpose	Example Questions
<code>def getHitterLeaderboard(df, hitters, metric, min_ab, start_date, end_date, top_n, ascending):</code>	Returns Dataframe of ranked hitter by specific metric	Who has the most homeruns?/Who has the most homeruns during the month of may?
<code>def getPitcherLeaderboard(df, pitchers, metric, min_ab, start_date, end_date, top_n, ascending):</code>	Returns Dataframe of ranked pitcher by specific metric	Which pitcher has the most strikeouts/ Who has the most strikeouts during the month of may?
<code>def getSingleHitterStats(df, hitters, hitter_name, start_date, end_date):</code>	Gets one specific hitter's stats	How much HR did Ohtani have?
<code>def getSinglePitcherStats(df, pitchers, pitcher_name, start_date, end_date):</code>	Gets one specific pitcher's stats	What was Yamamoto's ERA?
<code>def compareHitters(df, hitters, player1, player2, start_date=None, end_date=None):</code>	Compares two hitters	Compare Judge and Ohtani
<code>def comparePitchers(df, pitchers, player1, player2, start_date=None, end_date=None):</code>	Compares two pitchers	Compare Rodón and Yamamoto
<code>def getHitterLeaderboardVsPitcherHand(df, hitters, pitcher_hand="L", metric="BA", min_ab=50, top_n=10, ascending=False, start_date=None, end_date=None):</code>	Filters hitters by pitcher handedness	Who has the best BA vs lefties?

def hitterVsPitcher(df, hitters, hitter_name, pitcher_name):	Specific hitter vs pitcher matchup	How did Judge do against Yamamoto?
def getHitterStatsVsPitchType(df, hitters, pitch_name, hitter_name=None, metric="BA", min_ab=20, top_n=None, descending=False):	Hitter stats by pitch type	How did Judge do against sliders?

Challenges encountered during tooling Phase:

One of the biggest challenges in this project was working back and forth between multiple datasets. The project used Statcast data, Baseball Reference hitter data, and Baseball Reference pitcher data. Each dataset had different strengths. Statcast was useful for event-level questions, date filtering, pitch types, and matchups. The hitter and pitcher tables were useful for official full-season stats and player names. Matching these datasets together required using MLB player IDs and checking that the values lined up correctly.

Another issue was player names. Some Spanish names and accented characters were not displaying correctly at first. Names such as Acuña, José, Díaz, and Hernández appeared with broken encoding. This caused problems when searching for players by name, so I had to add a name-cleaning function to fix the encoding and make lookups more reliable.

A related problem was that many batter IDs from Statcast did not match cleanly to the hitter dataframe. At one point, there were hundreds of “Unknown Player” results. This happened because Statcast contained player IDs that were not always present in the hitter table being used. To avoid incorrect leaderboards, I removed unknown hitters from hitter-based results. For pitchers, I was more careful because pitcher names could still be pulled from the Statcast player_name column.

Another challenge was understanding which Statcast events should count as real baseball statistics. For example, truncated_pa appeared in the data. These are partial plate appearances that do not become completed official batting events. A batter may start a plate appearance, but the inning can end because of a caught stealing, pickoff, or another baserunning out. Because these are not completed plate appearances, I excluded truncated_pa from plate appearance and at-bat counts.

Strikeouts also had a special issue. At first, some pitcher strikeout totals were slightly off because Statcast has both strikeout and strikeout_double_play events. A strikeout double

play still includes a strikeout, so I added `strikeout_double_play` to the strikeout event list. This fixed missing strikeouts for pitchers like Carlos Rodón and Yoshinobu Yamamoto.

Pitching stats were also more complicated than hitting stats. Some stats, like strikeouts, hits allowed, walks allowed, and home runs allowed, could be calculated from Statcast. However, other stats like ERA, WHIP, innings pitched, wins, and losses were easier and more accurate to pull from the official pitcher table. Because of this, I added official pitcher stats only when no date filter is being used. For date-specific questions, the system uses Statcast-calculated stats only.

Another issue was deciding when to use official full-season data versus Statcast event data. For example, if the user asks, “How many walks did Max Muncy have?”, the official hitter table is simpler and reliable. But if the user asks, “How many walks did Max Muncy have in May?”, the system needs Statcast because the official hitter table does not support date filtering. This led to creating separate tools for full-season player stats and event-level analysis.

V.) Limitations

The main limitations of the overall project are:

- The project focuses on the 2025 MLB regular season.
- The notebook is the interface; no frontend was built for the class version.
- ERA and WHIP are official full-season stats and are not date-filtered.
- Pitch-type stats are based on plate appearances ending on that pitch type.
- Very advanced questions like year-over-year improvement are not fully supported.
- The LLM can sometimes choose the wrong tool if the question is ambiguous.
- Some answers depend on minimum sample-size filters.

VI.) Future Work

In the future, this project could be expanded into a full web application so users can ask baseball questions through a simple interface instead of using a notebook. This would make the system easier to use for announcers, analysts, and fans. The project could also be improved with more baseball tools, such as team-level stats, recent hot streaks, year-to-year comparisons, and more advanced matchup questions. Another useful addition would be charts or visualizations to show player trends, pitch-type results, and matchup performance more clearly. A longer-term goal would be to connect the system to live or near-real-time data, so it could be used during games when quick and specific information is needed.